High-Confidence Computing 5 (2025) 100314

Contents lists available at ScienceDirect

High-Confidence Computing

journal homepage: www.sciencedirect.com/journal/high-confidence-computing

Research article

Task migration with deadlines using machine learning-based dwell time prediction in vehicular micro clouds

Ziqi Zhou [a], Agon Memedi [a], Chunghan Lee [b], Seyhan Ucar [b], Onur Altintas [b], Falko Dressler [a,*]

[a] School of Electrical Engineering and Computer Science, TU Berlin, Berlin 10587, Germany
[b] InfoTech Labs, Toyota Motor North America R&D, Mountain View 94043, USA

ARTICLE INFO

Article history:
Received 22 November 2024
Revised 15 January 2025
Accepted 19 February 2025
Available online 3 March 2025

Keywords:
Edge computing
Vehicular micro cloud
Task migration
Task offloading
Dwell time prediction

ABSTRACT

Edge computing is becoming ever more relevant to offload compute-heavy tasks in vehicular networks. In this context, the concept of vehicular micro clouds (VMCs) has been proposed to use compute and storage resources on nearby vehicles to complete computational tasks. As many tasks in this application domain are time critical, offloading to the cloud is prohibitive. Additionally, task deadlines have to be dealt with. This paper addresses two main challenges. First, we present a task migration algorithm supporting deadlines in vehicular edge computing. The algorithm is following the earliest deadline first model but in presence of dynamic processing resources, *i.e*, vehicles joining and leaving a VMC. This task offloading is very sensitive to the mobility of vehicles in a VMC, *i.e*, the so-called dwell time a vehicles spends in the VMC. Thus, secondly, we propose a machine learning-based solution for dwell time prediction. Our dwell time prediction model uses a random forest approach to estimate how long a vehicle will stay in a VMC. Our approach is evaluated using mobility traces of an artificial simple intersection scenario as well as of real urban traffic in cities of Luxembourg and Nagoya. Our proposed approach is able to realize low-delay and low-failure task migration in dynamic vehicular conditions, advancing the state of the art in vehicular edge computing.

© 2025 The Author(s). Published by Elsevier B.V. on behalf of Shandong University. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

1. Introduction

Vehicle-to-everything (V2X) networking technologies have been in the focus of our research community for a long time [1]. Now, with the development of cellular V2X (C-V2X) [2,3] and mobile edge computing (MEC) in the context of 5G [4,5], new opportunities for advanced communication and computation among modern vehicles arise. C-V2X enhances communication reliability and scalability, while MEC deploys computing resources close to vehicles. These technologies help make the idea of vehicular micro cloud (VMC) [6,7] a reality. The concept of VMCs has been generalized in the form of virtualized edge computing [8], which is expected to be an integral part of 6G networks. In general, a VMC (or the virtualized edge) allows vehicles to utilize computing resources from other vehicles in close proximity. Migrating tasks within a VMC therefore helps decreasing task completion time cost and is particularly beneficial for delay-sensitive tasks. Compared to typical cloud-based technologies, VMCs serve as local edge servers, reducing communication delay. By clustering vehicles into small groups and leveraging the computational capacity of vehicles in the vicinity, VMCs enable more efficient resource allocation via task offloading.

Stationary and mobile vehicular micro clouds are two types of VMCs, formed by a certain geographical region or moving cars respectively. This paper focuses on the stationary VMC where vehicles enter a predefined region and offload tasks to other vehicles only in this area. This collaborative computing approach can significantly reduce task completion times by utilizing currently idle high computing capacity resources from neighboring vehicles. However, the dynamic nature of vehicular environments brings several challenges to achieving this efficiently. One of these is the mobility of vehicles, which can lead to task failures due to migration problems due to imprecise information about near-future position, and thus connectivity, of the involved vehicles. When either the user or the vehicle currently processing the task leaves the VMC, the task cannot be completed.

In this paper, we present a heuristic for optimized task migration in a VMC. This extends our previous work in [9]. Our heuristic task migration strategy minimizes the task-finishing time. We propose a greedy migration algorithm that selects the least time-cost destination for each task to be offloaded. Making use of task deadlines, this resembles an earliest deadline first (EDF) behavior in a dynamic and mobile context. Our heuristic makes use of a vehicle dwell time prediction model to determine an approximate

* Corresponding author.
E-mail address: dressler@ccs-labs.org (F. Dressler).

https://doi.org/10.1016/j.hcc.2025.100314
2667-2952/© 2025 The Author(s). Published by Elsevier B.V. on behalf of Shandong University. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

time the vehicle will still be part of the VMC. Specifically, we propose a machine learning (ML)-based dwell time prediction model using a random forest approach.

As an initial reference, we use an artificial single intersection to explore the performance of our heuristic. For proof of concept, we conducted simulations using realistic urban traffic data from the mid-scale European city Luxembourg [10], and the Japanese city Nagoya [11]. The task migration performance is evaluated using four metrics and a utility function that combines both task completion time and task failure rate, providing a view on practicability of the approach. It takes two types of task failure cases into account: (i) cars leaving the VMC before task completion, and (ii) task completion time exceeding the deadline.

This paper extends our previous work [9]. The main contributions beyond the current state of the art can be summarized as follows:

- We developed a novel task migration algorithm for use in vehicular micro clouds. The algorithm is based on a greedy approach, sequentially offloading tasks to the neighbor where it can be completed the earliest. In addition, deadlines are considered as a constraint. This matches the well-known EDF approach, now in a dynamic and mobile environment.
- In order to estimate the time a vehicle will be part of the VMC (the dwell time), we propose a new ML-based dwell time prediction model. In particular, we use a random forest algorithm that significantly outperforms previous heuristics.
- In a comprehensive simulation study, we thoroughly compare our task allocation algorithm and the dwell time prediction to other published heuristics and a theoretical optimum. Our results underline the efficacy of our solution.

The rest of our paper is organized as follows. We review the state-of-the-art related to both task offloading as well as vehicular predictions in Section 2. We present the system model, cost function, and utility function in Section 3. In Section 4, we introduce our proposed task migration algorithm. The ML-based dwell time prediction model is presented in Section 4.4. We discuss selected results from an extensive performance evaluation in Section 5. Finally, we present concluding remarks in Section 6.

2. Related work

In the following, we review related work for our main contributions: task offloading and dwell time prediction. To ease working with acronyms used, we summarize these in Table 1.

2.1. Task offloading

Previous work on task offloading has focused on various optimization strategies and optimization goals, including reducing energy consumption [12], minimizing latency [13], and monetary cost [14], and increasing task success rates [15]. In the following, we categorize these works based on their own optimization approaches: the combination of task offloading and resource allocation, learning-based, game-theoretic, or mobility-aware task offloading.

2.1.1. Task offloading and resource allocation

Mao et al. [15] proposed a joint optimization strategy considering the current system state, including transmitting energy and CPU resources, to reduce execution costs and task failures. Hossain et al. [13] introduced small-cell MEC to enhance offloading decisions and reduce task execution latency. Tran and Pompili [12] optimized a weighted sum of task completion time and energy consumption using convex and quasi-convex optimization techniques, while Zhao et al. [16] applied a distributed task-offloading algorithm for vehicles in MEC to minimize average delay. Although these approaches demonstrated improved resource efficiency, they primarily used simplified network environments and did not account for complex urban traffic dynamics, which limits their applicability in more realistic settings.

2.1.2. Learning-based approaches

Wu et al. [17] developed a hybrid task offloading scheme for vehicles using deep Q-learning (DQN) to minimize both task delay and energy consumption. Qin et al. [18] introduced a multi-vehicle task offloading algorithm using the multi-armed bandit (MAB) theory, which significantly reduced task delay and improved resource utilization. While these learning-based approaches showed considerable performance gains, they were validated in controlled environments that lacked the complexity of real urban traffic. Moreover, the lack of realistic mobility data in the training process may lead to significant deviations in performance when applied in practical scenarios.

2.1.3. Game-theoretic approaches

Zhao et al. [19] utilized a Nash equilibrium approach for optimal computation offloading and resource allocation, effectively reducing task processing delays and computation costs. Cheng et al. [14] proposed a pricing mechanism for resource allocation that minimized users' total costs while enhancing the monetary utility of the offloading process. These game-theoretic methods provided effective solutions for resource allocation; however, they were based on assumptions that may not hold in more dynamic environments.

2.1.4. Mobility-aware approaches

Cha et al. [20] designed a virtual edge formation framework to predict link duration between vehicles and reduce offloading failures due to unexpected disconnections. Zou et al. [21] proposed a collaborative task offloading strategy for vehicles using MEC, effectively reducing energy consumption by leveraging nearby edge resources. However, both approaches were validated in simplified scenarios, such as straight roads or unrealistic highways, which do not fully capture the complexity of urban traffic conditions.

Despite these advancements, challenges remain in task offloading, particularly in dynamic and unpredictable vehicular environments. In this regard, we advance the state of the art by integrating a more sophisticated mobility prediction model exploiting real urban data, to further improve task offloading efficiency, success, and applicability in real-world settings.

2.2. Dwell time and trajectory prediction

In dynamic vehicular environments, mobility prediction models can enhance the offloading process by forecasting relevant metrics such as vehicle trajectories and dwell time. Targeting dwell time estimation, Pannu et al. [22], Schettler et al. [23] utilized artificial neural network (ANN), random forest, and an empirical approach to predict vehicles' dwell time in the Luxembourg city [10] and in Manhattan grid scenarios. However, these models focus solely on dwell time prediction without integrating them into the task offloading or migration strategies.

Other offloading approaches focus on predicting vehicle trajectories [24], location forecasts [25], or overall traffic flow [26]. Xu et al. [26] propose a graph-weighted convolution network for traffic flow prediction. However, this approach diverges from our needs on migration algorithms, where precise dwell time estimation for individual vehicles in stationary VMCs, where the VMC region is fixed and pre-defined, is crucial. General traffic

Table 1
List of relevant acronyms.

Acronym	Description
ANN	artificial neural network
CPU	central processing unit
C-V2X	cellular V2X
DQN	deep Q-learning
EDF	earliest deadline first
FCFS	first-come first-served
ITS	intelligent transportation system
LSTM	long short-term memory
MAB	multi-armed bandit
MAE	mean absolute error
MB	megabyte
MEC	mobile edge computing
MI	million instructions
MIPS	million instructions per second
ML	machine learning
MSE	mean squared error
RF	random forest
SUMO	simulation of urban mobility
V2X	vehicle-to-everything
VMC	vehicular micro cloud

flow statistics do not provide the level of granularity we require for our dwell prediction model for each individual vehicle.

Guo et al. [24] develop a dual LSTM-based model to predict vehicle trajectories for improving task offloading in software-defined vehicular cooperative networks. However, the LSTM-based model is complex and requires significant computational resources for both training and inference. In contrast, our random forest model is simpler, requires less computational power, and is thus more practical for real-time deployment. Zhang et al. [25] use an extended Kalman filter to predict vehicle locations, relying on linear approximations that were not validated in a real-world urban traffic setting. Their experiments involved small unmanned vehicles operating indoors, which fails to capture the complex, non-linear mobility patterns typical of urban environments. In contrast, our dwell time prediction model is trained and predicts the realistic dwell time of vehicles effectively. Lv et al. [27] propose a DQN-based task offloading scheme in the handover area of vehicular edge computing networks based on trajectory prediction. Similarly, the complexity of a DQN model makes real-time decision-making challenging in dynamic vehicular environments, and the lack of real-world validation limits its robustness.

2.3. Open research gap and contributions

Most current research in vehicular task offloading neglects accurate dwell time prediction, which we will show in this paper is essential to ensure task completion without failure. Also, task failure rates are often overlooked as an evaluation metric, which is critical in vehicular environments, where tasks often have hard deadlines. To address these gaps, this work introduces a novel heuristic for task offloading approach aimed at minimizing task completion time and reducing task failures. We incorporate vehicle dwell time prediction using a random forest model, ensuring tasks are offloaded only when vehicles will remain within the VMC for the entire task duration. Our utility function optimizes both task completion time and task failure rates, making the solution closer to real-world requirements.

3. System model, cost function, and utility

3.1. System model

The system incorporates four elements: VMC, vehicles, tasks, and a controller. The VMC comprises dynamic vehicles in a certain region. Vehicles can act as both task generators and mobile edge servers with varying computational capabilities, enabling efficient task processing. Tasks generated by vehicles are heterogeneous in data size, computational complexity, and deadlines. The central controller predicts vehicular dwell times and prepares task migrations schedules. An example is illustrated in Fig. 1. There are five cars in this VMC and the tasks generated by cars have identical colors with the generator. Car A processes locally three generated tasks and migrates two tasks to Car B for remote processing. Similarly, Car C executes two local tasks and offloads three tasks to Cars D and E, respectively. Additionally, Car D migrates one task to Car C. Our task migration strategy re-arranges computing resource allocation and changes the task completion delay. For better readability see Table 2 summarizes all the used variables of our system model.

Table 2
Used variables.

Variable	Interpretation
a	the unit time interval
$Dwell(i,t)$	the estimated dwell time of $\mathsf{Veh}(i)$ at time t
$\mathcal{D}$	datasets of vehicular information
$\mathcal{D}_{train}$	training sets for ML
$\mathcal{D}_{test}$	testing sets for ML
d_{max}	Maximum tree depth for ML
δ	maximum local task completion time in the past
f_{max}	Maximum features for each split for ML
F	the constructed feature set of vehicles
$g(k)$	the generator vehicle of task k
i	the identification number of a car
j^*	the vehicle having the least workload in the set $\mathsf{OPT}(k)$
k	the identification number of a task
L	list of all newly generated tasks
L'	sorted list of all newly generated tasks
$MI(k)$	the complexity of task k
$MIPS(i)$	the computing capacity of car i
N_{trees}	Number of trees for ML
$\mathsf{OPT}(k)$	the set of processor vehicles with the minimum completion time for $\mathsf{Tsk}(k)$
$p(k)$	the processing vehicle of task k
$r(k)$	the result receiving vehicle of task k
R	transmission data rate
$S(k)$	the data size of task k
s_{split}	Minimum samples required to split for ML
s_{leaf}	Minimum samples required at leaf node for ML
T_k	the decision tree in random forest learning for ML
$\mathsf{Tsk}(k)$	task k
t	the current time step
t_i^{enter}	the timestep when car i enters the VMC
t_i^{exit}	the timestep when car i exits the VMC
$t_{gen}(k)$	the generation time of task k
$t_{ddl}(k)$	the completion deadline of task k
$T^{\text{c}}(k,p(k))$	the migration time of task k from $g(k)$ to $p(k)$
$T^{\text{p}}(k,p(k))$	the processing time of task k
$T^{\text{w}}(k,t,p(k))$	the waiting time of task k to be processed in car $p(k)$ at time t
$T(k,j)$	the completion time for $\mathsf{Tsk}(k)$ executed by $\mathsf{Veh}(j)$
$\mathsf{Veh}(i)$	car i
Veh_t	set of vehicles in the current VMC
(x,y)	coordinates of vehicles
$(x,y)_t$	coordinates of vehicles at time t
X	the feature set of vehicles
$\hat{Y}$	the final prediction of all dwell times
Y	the accurate dwell time set

3.1.1. Vehicular micro cloud (VMC)

A VMC represents a collection of cars and their computational resources that are in close physical proximity [6,7]. All cars in a VMC can have multiple roles: They can act as mobile edge servers with diverse computational abilities, and as users requiring reliable and efficient task-processing services.

Three main characteristics of tasks generated by vehicles within the VMC significantly impact our migration performance:

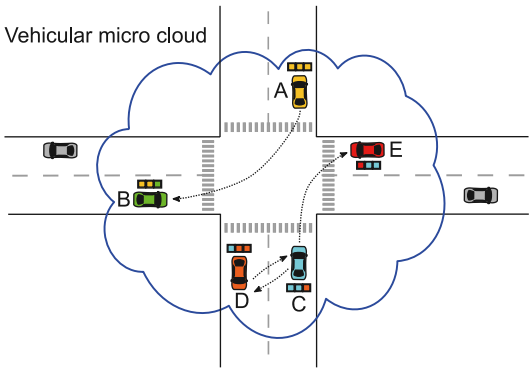


Fig. 1. Illustration of the VMC. Vehicles that participate in the VMC are color coded, with corresponding tasks and schedules. Each vehicle can act both as user and server. Vehicle A is processing most of its tasks locally and has two tasks offloaded to vehicle B. Vehicle C has offloaded its tasks to D and E, and it is processing a task of D.

task data size, execution complexity, and associated deadline. The task data size impacts the uplink and downlink delays between the task generator (user) and its remote processor (server); execution complexity influences the actual processing time; and the deadline indicates the maximum available computation and communication time.

3.1.2. Vehicle model

The presence of vehicles, their computing capabilities, and their current resource schedule in a VMC fluctuate over time, requiring migration decisions to be made at each time step to ensure optimal task assignment. We use t and i to denote time steps and individual cars, respectively. Thus, the tuple $(t_i^{\text{enter}}, t_i^{\text{exit}})$ indicates the time a car is available in the VMC. The total number of vehicles that ever entered the VMC at any given time is represented by $\max\{i\}$. As for the tasks generated by each vehicle at each time step, we assume this number follows a uniform distribution in $[0, n]$, and a central controller (per VMC) is assumed to receive all updates at each time step. Vehicles are assigned varying computing capabilities, measured in million instructions per second (MIPS). Some studies define computing power in terms of CPU cycles, but using MIPS provides a more straightforward measure since one cycle can execute multiple instructions.

3.1.3. Task model

We use k to denote the sequence number of a task. Thus, $\max\{k\}$ represents the total number of tasks generated by vehicles that have been part of the VMC. Tasks are heterogeneous in terms of data size, execution complexity, and deadlines. To ensure compatibility with the vehicles' computing capacities, we measure task complexity in million instructions (MI). Additionally, task data sizes indicate the amount of data that must be transmitted for task execution, measured in megabytes (MB). We assume task deadlines are uniformly distributed. The overall delay T_k of task k comprises communication T_k^{c}, waiting for execution T_k^{w}, and processing T_k^{p} where $T_k \triangleq T_k^{\text{w}} + T_k^{\text{c}} + T_k^{\text{p}}$. For simplicity, we assume that no congestion or collision occurs in communication.

3.1.4. Controller model

The central (per VMC) controller is responsible for collecting information, listing tasks according to their deadlines, calculating potential task completion times, training vehicular dwell time prediction models, predicting dwell times, and making task migration decisions. Without loss of generality, we assume such a controller always exists. This role can be carried out by one of the cars in the VMC, and handed over if the car leaves.

Information collection and dwell time prediction involves the following steps: (1) For each time slot t, the central controller adds newly joined vehicles (including current location, speed, type, lane, task queue, and computing capacity) to the vehicle set. (2) When a vehicle has left the VMC during the previous time slot $t-1$, the controller removes it from the set. Furthermore, its dwell time is recorded and added to the dataset used to train the dwell time prediction model. (3) For all vehicles in the VMC, the dwell time is predicted to be used in migration decisions. (4) When a vehicle generates new tasks, the central controller collects task information, including data size, computational complexity, and deadlines.

3.2. Preliminaries

Let $\mathsf{VMC} = \{\mathsf{Veh}, \mathsf{Tsk}\}$ be a vehicular micro cloud with Veh being the set of all vehicles and Tsk being the set of all tasks. $\mathsf{Veh}(i)$ represents the ith vehicle in Veh and $\mathsf{Tsk}(k)$ is the kth task in Tsk.

For each $\mathsf{Veh}(i)$, we denote the VMC entering time $t_{\text{enter}(i)}$, VMC exiting time $t_{\text{exit}}(i)$, and its computing capability $\mathsf{MIPS}(i)$. Thus,

$$\mathsf{Veh}(i) = (t_{\text{enter}(i)}, t_{\text{exit}}(i), \mathsf{MIPS}(i)). \tag{1}$$

For each $\mathsf{Tsk}(k)$, we denote its generation time $t_{gen}(k)$, computing complexity $\mathsf{MI}(k)$, data size $S(k)$, its generator vehicle index $g(k)$, its processing vehicle index $p(k)$, completion deadline $t_{ddl}(k)$, and its result receiver vehicle $r(k)$.[1] That is

$$\mathsf{Tsk}(k) = (t_{\text{gen}}(k), \mathsf{MI}(k), S(k), g(k), p(k), t_{ddl}(k), r(k)). \tag{2}$$

where $p(k)$ and $r(k)$ can only be known when the task migration has been decided.

Let $T^{\text{c}}(k, p(k))$ be the uplink time of $\mathsf{Tsk}(k)$ migrating from $\mathsf{Veh}(g(k))$ to $\mathsf{Veh}(p(k))$ and the channel data rate be R. Then for any $\mathsf{Tsk}(k)$, we have

$$T^{\text{c}}(k, p(k)) = \begin{cases} 0, & \text{if } g(k) = p(k) \\ \frac{S(k)}{R}, & \text{otherwise.} \end{cases} \tag{3}$$

In our communication model, as long as $g(k) \neq p(k)$, the communication time remains constant.

Let $T^{\text{p}}(k, p(k))$ be the task processing time cost $\mathsf{Tsk}(k)$ processed by $\mathsf{Veh}(p(k))$,

$$T^{\text{p}}(k, p(k)) = \frac{\mathsf{MI}(k)}{\mathsf{MIPS}(p(k))} \tag{4}$$

Let $T^{\text{w}}(k, t, p(k))$ be the duration of $\mathsf{Tsk}(k)$ waiting for its execution by $\mathsf{Veh}(p(k))$ at time t. The waiting task queue in $\mathsf{Veh}(p(k))$ is $\mathsf{Tsk}(k_0), \mathsf{Tsk}(k_1), \ldots, \mathsf{Tsk}(k_q)$, where $\mathsf{Tsk}(k_0)$ is under processing and the other tasks are waiting for execution. Then

$$T^{\text{w}}(k, t, p(k)) = \frac{(1 - \alpha_{p(k),t})\mathsf{MI}(k_0)}{\mathsf{MIPS}(p(k))} + \sum_{n=1}^{q-1} T^{\text{p}}(k, p(k)), \tag{5}$$

where $\alpha_{i,t}$ implies the proportion of instructions that have been finished of $\mathsf{Tsk}(k_0)$.

[1] To clarity, $r(k)$ represents the car to which the result of the generated task is returned. This can either be the original task-generating car, indicating a successful result return, or remain empty, signifying a failed return.

3.3. Cost function and utility function

Our optimization problem can be formulated in a simplified form by minimizing a time cost function without accounting for task failure, or more comprehensively by maximizing a utility function. The time cost function aims to minimize the total task completion time. The utility function is designed to balance multiple factors by combining the average task completion time with two weighted metrics for task failure: the proportion of tasks that miss their deadlines and the proportion of tasks where the generating or processing device exits the VMC before task completion.

We assume a $\mathsf{Tsk}(k)$ is generated and migrated at time t to $\mathsf{Veh}(p(k))$, where there have been already $q-1$ tasks in the waiting queue and $\mathsf{Tsk}(k)$ is the qth task in the queue.

The expression $T^{\text{sum}}(k,t,p(k))$ represents the completion time of task $\mathsf{Tsk}(k)$ as

$$\begin{aligned} T^{\text{sum}}(k,t,p(k)) =& T^{\text{c}}(k,p(k)) + T^{\text{p}}(k,p(k)) + T^{\text{w}}(k,t,p(k)) \\ =& \mathbb{I}(p(k)\neq g(k))\frac{S(k)}{R} + \frac{\mathsf{MI}(k)}{\mathsf{MIPS}(p(k))} + \\ & \frac{(1-\alpha_{p(k),t})\mathsf{MI}(k_0)}{\mathsf{MIPS}(p(k))} + \sum_{n=1}^{q-1} T^{\text{p}}(k_n) \end{aligned}$$

It consists of communication time $T^{\text{c}}(k,p(k))$, processing time $T^{\text{p}}(k,p(k))$, and the waiting time $T^{\text{w}}(k,t,p(k))$. The communication cost is positive only if the task is not executed locally, *i.e*, $p(k)\neq g(k)$. Therefore, we have

$$\begin{aligned} T^{\text{sum}}(k,t,p(k)) =& \mathbb{I}(p(k)\neq g(k))\frac{S(k)}{R} + \\ & \sum_{n=1}^{q}\frac{\mathsf{MI}(k_n)}{\mathsf{MIPS}(p(k))} + \frac{(1-\alpha_{p(k),t})\mathsf{MI}(k_0)}{\mathsf{MIPS}(p(k))} \end{aligned} \tag{6}$$

The optimization problem can now be formulated as

$$\begin{aligned} &\underset{k}{\text{minimize}} \quad T^{\text{sum}}(k,t,p(k)) \\ &\text{subject to} \quad t\geq 0, \quad k,p(k),q\geq 1 \end{aligned} \tag{7}$$

The sum of completion time for N tasks over a certain duration δ is

$$T^{\text{sum}}(N,\delta) = \sum_{k=1,t=0}^{k=N,t=\delta}(k,t,p(k)) \tag{8}$$

Before introducing the utility function, the failure rate (caused by unpredicted leaving vehicles) $F_m(N,\delta)$, and the failure rate (caused by missing task deadlines) $F_d(N,\delta)$ over a certain time interval δ with N tasks generated in VMC are respectively defined as:

$$\begin{aligned} F_m(N,\delta) &= \frac{\sum_{k=1}^{N}\mathbb{I}(r(k)\neq g(k))}{N} \\ F_d(N,\delta) &= \frac{1}{N}\sum_{k=1}^{N}\mathbb{I}(T^{\text{sum}}(k,t,p(k)) > (t_{ddl}(k)-t_{gen}(k))) \end{aligned} \tag{9}$$

Moreover, these two task failure rates are independent. For instance, if a task is completed before its deadline but its generator has left before it is completed, then the task failure can only be accounted into F_m, but not into F_d.

The overall utility function can now be formulated as

$$\begin{aligned} U(N,\delta) =& -\frac{1}{N}T^{\text{sum}}(N,\delta) - \beta\frac{\sum_{k=1}^{N}\mathbb{I}(r(k)\neq g(k))}{N} - \\ & \sigma\frac{\sum_{k=1}^{N}\mathbb{I}T^{\text{sum}}(k,t,p(k)) > (t_{ddl}(k)-t_{gen}(k))}{N} \end{aligned} \tag{10}$$

where β and σ are the penalty coefficients of tasks being unsuccessfully returned or missing deadlines, respectively.

By minimizing the cost function, we determine the least completion time for a single task. Our approach uses a greedy heuristic that evaluates all potential processors, including the task generator, to offload the task to the least time-consuming option. This procedure is repeated for each subsequent task to achieve the shortest offloading time. The primary difference between the cost function and the utility function lies in their scope: the cost function focuses on minimizing the completion time for a single task, forming the foundation for the system utility function. In contrast, the utility function considers both delay and failure costs for all tasks within the VMC over a specified duration. Locally processed tasks are guaranteed to be completed without failure caused by unpredictably leaving vehicles, though they may still miss deadlines.

4. Comprehensive task migration algorithms

We designed a task migration algorithm taking the following objectives into consideration. First, we aim to realize the sequence of tasks being selected for offloading based on their deadlines (Algorithm 1). Second, the central controller steers the offloading process, taking dynamic vehicular dwell time prediction into account (Algorithm 2). Last and essentially, we identify the best vehicle for task migration based on a pre-sorted sequence (Algorithm 3), especially by leveraging future dwell times to reduce the probability of task failures (Algorithm 4).

4.1. Task pre-scheduling

The task pre-scheduling function is applied only to tasks generated in the last time step. It prioritizes urgent tasks based on their deadlines, similar to the earliest deadline first (EDF) approach, but excludes tasks generated from previous time steps.[2] By reordering tasks, those with more urgent deadlines are prioritized over those with less strict deadlines. Algorithm 1 depicts all the task pre-scheduling steps.

Algorithm 1 Task pre-scheduling

Input: L: List of all newly generated tasks
Output: L': Sorted list of tasks by deadline
1: **function** PRE-SCHEDULE-TASKS(L)
2: $L \leftarrow$ collect tasks generated by vehicles in VMC at time t
3: $L' \leftarrow$ sort L in increasing order of deadlines
4: **return** L'
5: **end function**

4.2. Central task scheduling

The central task scheduling (Algorithm 2) is designed to optimize task assignment decisions in a VMC by predicting the remaining dwell time of vehicles and leverages the prediction result to dynamically offload tasks to the most appropriate vehicles for remote execution.

4.3. Task migration

The task migration algorithm concretely manages task offloading based on an ordered list of undecided tasks (Algorithm 3). The controller selects suitable processing cars based on the shortest task completion time within their dwell time constraints and with the least workload. If the original vehicle offers the shortest completion time, the task is assigned for local task processing.

[2] Please note that we do not support preemption and re-migration at the moment.

Algorithm 2 Central scheduling

Input: Set of vehicles Veh_t in the current VMC
Output: Predicted dwell times and task migration decisions
for each time step $t = 1, \ldots, T$ **do**
  for each vehicle $Veh(i)$ in the set of Veh_t **do**
    Predict dwell time of $Veh(i)$ using Algorithm 4
  end for
  for each vehicle $Veh(i)$ in the set of Veh_t **do**
    Execute task migration using Algorithm 3
  end for
end for

Algorithm 3 Task migration

Input: Initial vehicle set Veh, task set Tsk
Output: Optimized task assignment decisions
for each time slot $t = 1, 2, \ldots$ **do**
  for each vehicle $\mathsf{Veh}(i)$ in latest Veh set **do**
    Collect predicted dwell time $Dwell(i)$ of $\mathsf{Veh}(i)$
    Collect new and already assigned tasks
  end for
  Controller forms list of unassigned tasks L
  $L' \leftarrow$ sorted list of L from Algorithm 1
  for each task $\mathsf{Tsk}(k)$ in L' **do**
    for each vehicle $\mathsf{Veh}(j)$ in Veh **do**
      if $T(k,j) \le Dwell(i)$ **then**
        $T(k,j) \leftarrow T^{\mathrm{c}}(k,j) + T^{\mathrm{p}}(k,j) + T^{\mathrm{w}}(k,t,j)$
      end if
    end for
    $\mathsf{OPT}(k) \leftarrow \mathrm{argmin}_j\, T(k,j)$
      ▷ Find vehicles with minimum completion time for $\mathsf{Tsk}(k)$
    if $g(k) \in \mathsf{OPT}(k)$ **then** ▷ $g(k)$ is preferred vehicle for $\mathsf{Tsk}(k)$
      $p(k) \leftarrow g(k)$
    else
      Find $j^* \in \mathsf{OPT}(k)$ with minimum workload ▷ Select vehicle with least load
      $p(k) \leftarrow j^*$
    end if
  end for
end for

4.4. ML-based dwell time prediction

Accurately predicting the entire dwell time of vehicles from entry to exit in VMC has been shown challenging in [22,23]. We propose a random forest regression model (cf. Algorithm 4) to estimate only the future remaining time of cars, using previous location information and an adaptive upper bound. The model predicts the future dwell time that will suffice for task completion, disregarding any exceeding time longer than needed. By collecting the maximum task completion time denoted as δ over a time window $[t - \Delta, t]$, the upper bound δ serves as an adaptive threshold. Used features include the current time t, current position (x, y), and past positions sampled at time steps up to δ, ensuring sufficient temporal context for accurate predictions.

Our proposed random forest model predicts the future dwell time of each vehicle at each time step. Initially, it prepares a set of features, including the current time and its previous and current location, the previous time has threshold δ, and then it splits the dataset into training and testing sets. Key hyperparameters of the random forest model are N_{trees}: number of trees; d_{max}: maximum tree depth; s_{split}: minimum samples required to split; s_{leaf}: minimum samples required at leaf node; f_{max}: maximum features for each split. We used the Optuna hyperparameter optimization framework[3] for training and optimization.

During training, the algorithm creates multiple decision trees using bootstrapped samples. Each tree in the forest maps the input features (time and locations from δ seconds ago till now) to the future dwell time. The random forest model then makes predictions $\hat{y}$ by averaging the future dwell time outputs from all decision trees. The algorithm evaluates the model performance by mean squared error (MSE) and mean absolute error (MAE). Finally, the trained model is saved for future dwell time predictions.

Algorithm 4 Random forest dwell time prediction

Input: Floating car data collected over a certain time duration T
Output: Predicted future dwell time $Dwell(i)$ for each vehicle at each time step
$\delta \leftarrow$ maximum local task completion time in the past
$a \leftarrow$ interval between two time steps ▷ we use $a = 0.2$ s in this paper
$t \leftarrow \{t - T, t - T + a, t - T + 2a \ldots, t - 2a, t - a, t\}$, where t is the current time step
for each vehicle $Veh(i)$ **do**
  for each time step t in the duration of $[t - \delta + a, t]$ **do**
    Extract coordinates (x, y) for $t - a$
  end for
  Calculate the dwell time of $Veh(i)$ at t and upper bound it by δ
end for
Feature preparation:
Construct feature set $F = \{t, (x,y)_t, (x,y)_{t-a}, \ldots, (x,y)_{t-\delta}\}$
Split dataset $\mathcal{D}$ into training $\mathcal{D}_{train}$ and testing $\mathcal{D}_{test}$ sets
Model initialization and hyperparameter tuning:
Define random forest hyperparameters (N_{trees}, d_{max}, s_{split}, s_{leaf}, f_{max}) to optimize with Optuna
Training process:
for $k = 1, \ldots, N_{trees}$ **do**
  Train decision tree T_k on the training set $\mathcal{D}_{train}^{(k)}$
end for
The final prediction $\hat{Y}$ is

$$\hat{Y} = \frac{1}{N_{trees}} \sum_{k=1}^{N_{trees}} T_k(F)$$

The predicted future dwell time of $Veh(i)$ is

$$Dwell(i) = \hat{Y}_i$$

Model evaluation:
Compute MSE and MAE on $\mathcal{D}_{test}$:

$$\mathrm{MSE} = \frac{1}{|\mathcal{D}_{test}|} \sum_{(X_i, Y_i)} \in \mathcal{D}_{test} (Y_i - \hat{Y}_i)^2$$

$$\mathrm{MAE} = \frac{1}{|\mathcal{D}_{test}|} \sum_{(X_i, Y_i)} \in \mathcal{D}_{test} |Y_i - \hat{Y}_i|$$

Model deployment:
Save trained model f_{RF} for future predictions

[3] https://github.com/optuna/optuna

Table 3
Performance metrics of dwell time prediction across different models and scenarios (time step 0.2 s).

Model	Scenario	Kind	MSE [s^2]	MAE [s]
Proposed	Single intersection	Low	0.003	0.015
		Medium	0.006	0.028
		High	0.008	0.037
Proposed	Luxembourg	Suburb	0.008	0.046
		Highway	0.021	0.110
		City	0.008	0.046
Proposed	Nagoya	Suburb	0.066	0.027
		State	0.030	0.026
		City	0.013	0.014
Johnson SU	Single intersection	Low	552.727	18.976
		Medium	491.971	17.680
		High	648.250	18.552
Johnson SU	Luxembourg	Suburb	0.615	0.633
		Highway	1177.998	27.880
		City	790.667	17.157
Johnson SU	Nagoya	Suburb	953.707	28.740
		State	690.588	21.335
		City	367.292	15.387

We validate our random forest model for dwell time estimation using three scenarios (cf. Section 5.1). Using the Optuna hyperparameter optimization framework, our proposed model achieves a MSE of less than 0.06 and a MAE of less than 0.03 s. In comparison, the Johnson SU distribution performs less effectively with a MSE of up to 1177 s^2 and a MAE of up to 29 s, which is in line with the results reported in [23]. The performance comparison is presented in detail in Table 3.

4.5. Example of operation and underlying conflicts

For further clarification, concretely, all newly generated tasks are ordered at each step based on their generation time. For instance, at time step t, tasks generated within the interval $(t-a, t]$ are considered.[4] Let us assume that only two tasks are generated during the interval $(t-a, t]$: task k, generated at $t-a+0.001$ seconds with a deadline of $t+2$ seconds, and task $k+1$, generated at $t-a+0.002$ seconds with a deadline of $t+1$ seconds. Initially, the tasks are sequentially ordered according to their generation times in the list L. However, as described in Algorithm 1, tasks generated during $(t-a, t]$ are re-sorted in the list L' at time t based on their priority, determined by their deadlines. Consequently, task $k+1$ is prioritized for offloading, as it has an earlier deadline.

In the decision-making process, the selected task's $k+1$ completion time is calculated for each potential offloading option. This time contains the communication time (only required for remote offloading), the waiting time for execution on a potential processor vehicles, and the processing delay give the different computing resources. Next, task $k+1$ is offloaded to the vehicle that minimizes its completion time. Meanwhile, the remaining computing resources are updated. Possible completion times of task k are calculated and it is offloaded accordingly. When an underlying conflicts occur, *i.e*, multiple tasks have the same deadline in the same time slot, the one that has been generated earlier will be prioritized in the list L'.

5. Simulation setup and results

In the following, we first present the simulation setup we used followed by an in-depth look at selected results.

[4] Here, a represents the interval between time steps, which is set to 200 ms in our experiments.

Table 4
Typical task properties used in the literature. If a range is given, a uniform distribution is assumed.

Reference	Data size [MB]	Complexity [MI]	CPU [MIPS]
Chen et al. [28]	0.5	1000	500 to 10 000
Jang et al. [29]	1.25 to 3.125	–	–
Mao et al. [15]	0.125	0.7375	1500
Liu et al. [30]	0.1 to 0.3	100 to 400	1000 to 5000
Bute et al. [31]	0.0125 to 0.125	1000 to 5000	1000 to 4000
Shi et al. [32]	0.02 to 0.2	200 to 3200	5000 to 10 000
Our evaluation	0.1 to 1	1000 to 5000	1000 to 5000

The – indicates that the detail has not been specified.

Table 5
Road traffic parameters: simple intersection scenario.

Parameter	Low	Med	High
Total number of cars	480	900	1200
Average speed [km/h]	27.86	25.74	21.20
Average dwell time [s]	865.16	1002.77	887.46
Simulation duration [s]	3600	3600	3600
Car generation period [s]	7.5	4	3
Tracking Time Step [ms]	200	200	200

5.1. Simulation parameters

Selecting representative parameters for task settings remains a challenge. This includes task types, data size, deadlines, and computing resource requirements. Following a literature study, we found that there is a huge variety of applications considered. Table 4 gives an overview of typical settings. If a range is given, a uniform distribution is assumed. Based on this overview, we selected the parameters for our performance evaluation (last row in the table). In addition, we assume task deadlines in the range 1 s to 1.5 s, which conceptually matches our time slots.

Without loss of generality and also following the observations in [9], we set the data rate to 50 Mbit s^{-1}, which can be easily supported by most current communication technologies. For accurate mobility simulation, we used SUMO Version 1.19.0.[5] [33] For best comparability, we use three different traffic scenarios.

5.1.1. Simple intersection

For ease of validation, we use a single intersection scenario. This scenario contains a single traffic light, four entry points, and 16 possible routes, including straight paths, right turns, left turns, and U-turns. The map covers an area of approximately 400 m × 400 m. Trips were generated using the SUMO random trip generator, which allows for flexible control over car generation periods, thereby affecting car density. After generating mobility for three different traffic densities (low, medium, high), the simulation outputs floating car data, including position, speed, lane, and other information about every vehicle at each time step in the simulation. Important characteristics such as the total number of cars, average speed, and average duration of cars can be analyzed and obtained from the traces output. The key parameters are summarized in Table 5.

5.1.2. Luxembourg scenario

The Luxembourg SUMO Traffic (LuST) Scenario [10] is a highly cited, detailed, and realistic traffic simulation model for the mid-sized European city of Luxembourg. This scenario provides an ideal environment for testing and evaluating the efficiency and robustness of our algorithm because of its complexity and representativeness of real-world urban traffic conditions. To assess

[5] https://eclipse.dev/sumo/

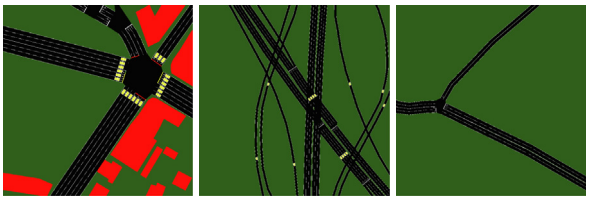


Fig. 2. Luxembourg scenario (a) City (b) Highway (c) Suburban.

Table 6
Road traffic parameters: Luxembourg scenario.

Time	Parameter	City	High-way	Sub-urban
Midnight	Total number of cars	5	8	0
00:20	Avg. speed [km/h]	16.778	87.846	0
	Avg. dwell time [s]	28.400	16.375	0
	Max. dwell time [s]	77.000	36.000	0
	Simulation time [s]	100.000	100.000	100.000
	Time Step [ms]	200.000	200.000	200.000
Morning	Total number of cars	102	168	10
08:20	Avg. speed [km/h]	8.040	89.920	52.730
	Avg. dwell time [s]	37.921	9.125	10.000
	Max. dwell time [s]	99.000	41.000	20.000
	Simulation time [s]	100.000	100.000	100.000
	Time Step [ms]	200.000	200.000	200.000
Afternoon	Total number of cars	68	64	2
16:20	Avg. speed [km/h]	11.826	104.970	43.077
	Avg. dwell time [s]	29.191	8.828	10.500
	Max. dwell time [s]	75.000	40.000	15.000
	Simulation time [s]	100.000	100.000	100.000
	Time Step [ms]	200.000	200.000	200.000

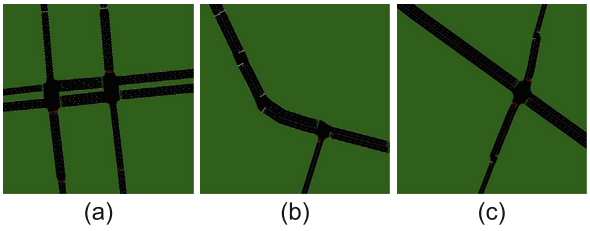


Fig. 3. Nagoya scenario (a) City (b) State road (c) Suburban.

the effectiveness of our migration mechanism in real traffic, we selected three distinct regions within the map as VMC zones, each characterized by varying vehicle flow and density. These zones are strategically located in the city center, on a highway, and in a suburban area (Fig. 2). For the SUMO simulations, we chose three representative time periods throughout the day: midnight, morning peak hour, and non-peak afternoon, each reflecting distinct traffic conditions. The key parameters are summarized in Table 6.

5.1.3. Nagoya scenario

We also selected the Nagoya Urban Mobility (NUMo) scenario [11] as one of the most comprehensive urban traffic scenarios, providing a highly realistic vehicle mobility of the entire city of Nagoya, Japan. Featuring a densely populated urban environment with complex traffic patterns, NUMo enhances our vehicular mobility simulations by reflecting a more accurate representation of real-world dynamics compared to other scenarios. NUMo's traffic signals are meticulously calibrated, and its primary data source is actual traffic counts. This makes NUMo an even closer approximation to real-world conditions, making it particularly valuable for our research. Three distinct regions within the map are chosen as VMC zones. These zones are located in the city center, on an entrance of a state road, and in a suburban area (cf. Fig. 3). For the SUMO simulations, we again chose three representative times of the day: midnight, morning peak hour, and non-peak afternoon, each reflecting distinct traffic conditions. The key parameters are summarized in Table 7.

Table 7
Road traffic parameters: Nagoya scenario.

Time	Parameter	City	State Road	Sub-urban
Midnight	Total number of cars	7	7	3
00:20	Avg. speed [km/h]	20.629	25.659	16.605
	Avg. dwell time [s]	25.571	21.000	22.000
	Max. dwell time [s]	33.000	59.000	43.000
	Simulation time [s]	100.000	100.000	100.000
	Time Step [ms]	200.000	200.000	200.000
Morning	Total number of cars	70	48	28
08:20	Avg. speed [km/h]	24.864	22.976	19.090
	Avg. dwell time [s]	17.028	31.520	27.285
	Max. dwell time [s]	58.000	67.000	92.000
	Simulation time [s]	100.000	100.000	100.000
	Time Step [ms]	200.000	200.000	200.000
Evening	Total number of cars	21	13	8
20:20	Avg. speed [km/h]	24.802	18.205	17.242
	Avg. dwell time [s]	24.809	41.076	31.000
	Max. dwell time [s]	50.000	66.000	47.000
	Simulation time [s]	100.000	100.000	100.000
	Time Step [ms]	200.000	200.000	200.000

5.2. Baselines for performance evaluation

5.2.1. No migration

In this approach, tasks are processed locally without any offloading or migration to other vehicles in the same VMC. By comparing task completion times between the *no migration* strategy and migration mechanisms, we can quantify the performance gains achieved through task migration strategies.

5.2.2. Migration based on accurate dwell time

All migration strategies use the same offloading algorithm but rely on different sources for future vehicular dwell times. This baseline assumes accurate knowledge of the dwell time, obtained from the SUMO traces. It is representing the theoretical optimal performance achievable.

5.2.3. Johnson SU distribution of dwell times

The Johnson SU distribution has been empirically identified as a good representation of dwell time distribution in Luxembourg (cf. [22]). Schettler et al. [23] already used it as a baseline in their research. Unlike other prediction approaches, this method does not require training a machine learning model.

5.3. Simple intersection scenario

Fig. 4 shows the performance of the four task offloading strategies for the simple intersection scenario. We plot the task completion time in Fig. 4(a) and the saved time ratio, *i.e*, the proportion of time saved by using three migration strategies compared to the no migration strategy, in Fig. 4(b).

Task completion time, defined as the duration from a task's generation to its completion – whether executed entirely within the VMC or partially outside it – is a straightforward and direct metric. In our setup, each car uniformly randomly generates 0 to 3 tasks per time-step. For *no migration*, the high-, medium-, and low-density scenarios show a descending order of completion times, which is reasonable. All migration approaches, whether using accurate knowledge or dwell time prediction models, demonstrate a significant decrease in completion delay compared to the

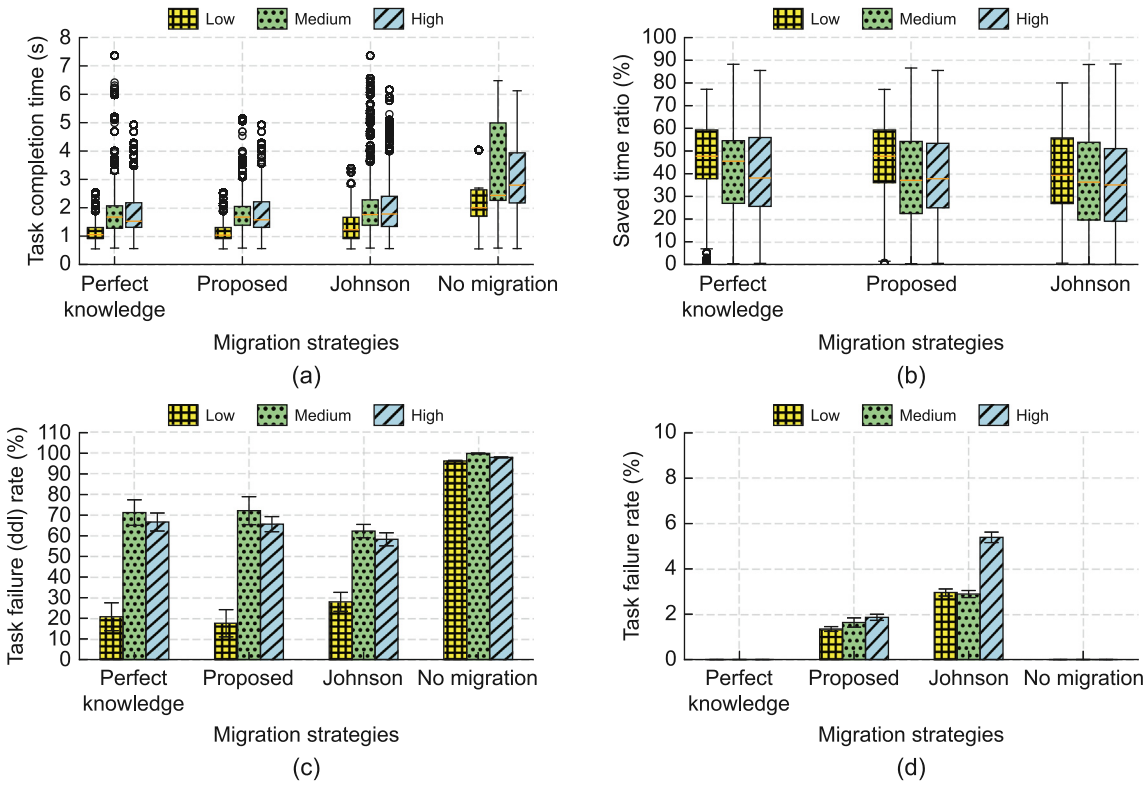


Fig. 4. Migration performance, simple intersection scenario. (a) Task completion time (b) Saved time ratio (c) Task failure rate due to missed deadline (d) Task failure rate due to mobility prediction.

no migration strategy. The benefit increases with increasing traffic density, *i.e*, increasing candidates for task migration.

The saved time ratio $\mathrm{T}^{\mathrm{save}}(k,t,p(k))$ is calculated as the proportion of the time difference between *no migration* and migration strategies, divided by *no migration* time as

$$\mathrm{T}^{\mathrm{save}}(k,t,p(k)) = \frac{T^{\mathrm{local}}(k,t,p(k)) - T^{\mathrm{mig}}(k,t,p(k))}{T^{\mathrm{local}}(k,t,p(k))}$$

It effectively illustrates the time-saving benefits of task offloading. Regardless of the offloading methods used, the proportions of saved time are similar. Additionally, the medium- and high-density scenarios show a higher proportion of tasks saving around 40 to 50% of the time when migrations using accurate dwell time knowledge or using our random forest model. In contrast, migration decisions based on Johnson SU distribution leads to only below 40% of the saved time ratio.

Fig. 4(c) shows the task failure rate due to missing deadlines, where a task result is completed after its deadline. Fig. 4(d) illustrates the task failure rate due to wrong predictions on cars' mobility, which occurs when the task generator or its processor car leaves the VMC before the task is completed. Both are two independent events. As can be seen, our proposed solution always outperforms Johnson distribution across all scenarios. No migration leads to a high task failure rate due to missing deadlines, and Johnson SU prediction leads to higher failure rates due to wrong mobility prediction.

5.4. Luxembourg scenario

For the Luxembourg scenario, we selected the morning rush hour data for discussion in this paper. During the non-peak period in the afternoon at 4 pm, the algorithm operates similarly to the peak hour. However, during midnight, only very few vehicles are present, limiting task processing to local execution. So, we concentrate on the different locations in the city. The results of our simulations for average task completion time and saved time ratio as well as the task migration failure rate are shown in Fig. 5.

In terms of reducing task completion time, all migration strategies are effective. The results for our proposed algorithm and for the theoretical optimum using perfect dwell time knowledge yield nearly identical performance, demonstrating the superiority of our solution. Again, the Johnson SU based task mitigation results in slightly increased task completion times (Fig. 5(a)) and lower saved time ratios (Fig. 5(b)), but the differences are not significant. However, when evaluating the task failure rate, Johnson SU leads to a pronounced failure rate, whereas our prediction model reduced this by about 36%.

5.5. Nagoya scenario

Also for the Nagoya scenario, we selected the morning rush hour data for discussion in this paper. During the non-peak periods in the evening at 8 pm, the algorithm operates similarly to the peak hour. During midnight, in the city and suburban areas of Nagoya, only one vehicle is present. As a result, the migration strategy requirements are not effective. Thus, again, we focus on the morning rush hour. The results of our simulations for average task completion time and saved time ratio as well as the task migration failure rate are shown in Fig. 6.

Task completion times (cf. Fig. 6(a)) across the three migration strategies are significantly reduced compared to no migration (cf. Fig. 6(b)). Our proposed model clearly outperforms the Johnson SU approach w.r.t. the task failure rate (mobility reason, cf. Fig. 6(d)): Our proposed solution exhibits a failure rate of around 0.5 % to 1.5 %, whereas the Johnson SU approach leads to 2.0 % to 5.0 % across all locations in Nagoya. This demonstrates the robustness and effectiveness of our migration mechanism when combined with the dynamic prediction model.

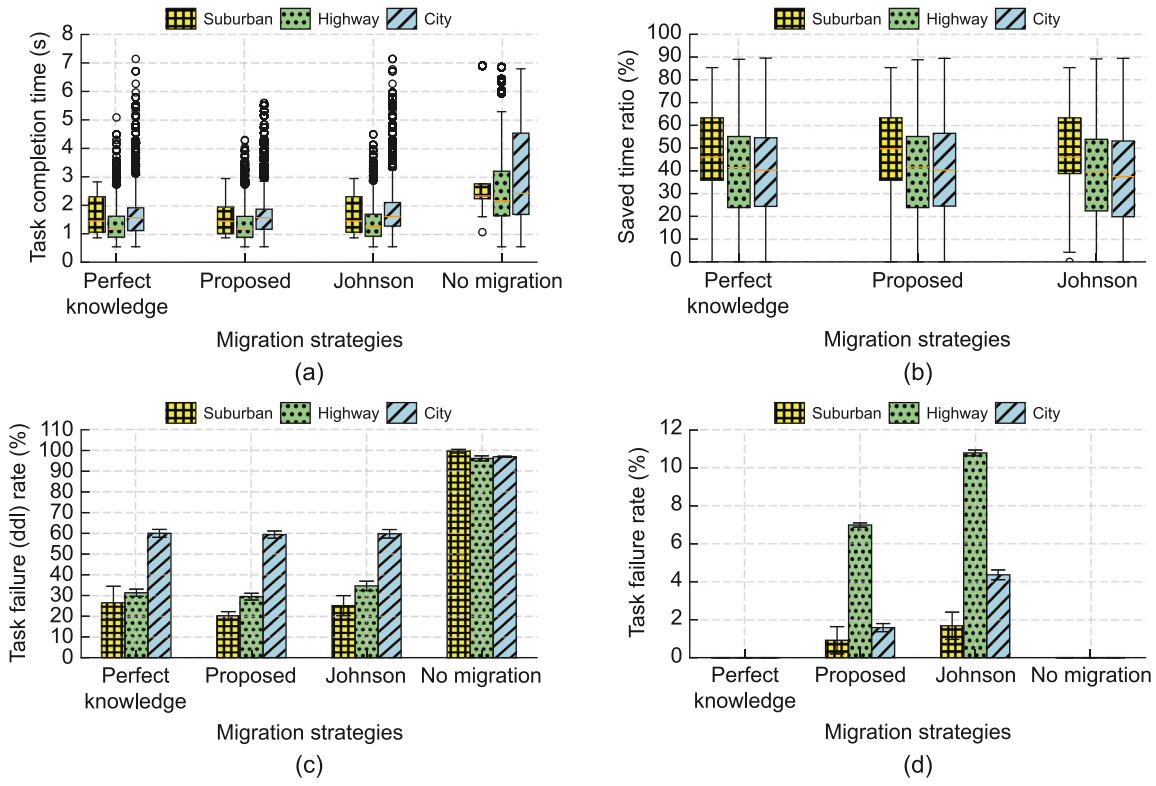


Fig. 5. Migration performance, Luxembourg scenario during morning rush hour. (a) Task completion time (b) Saved time ratio (c) Task failure rate due to missed deadline (d) Task failure rate due to mobility prediction.

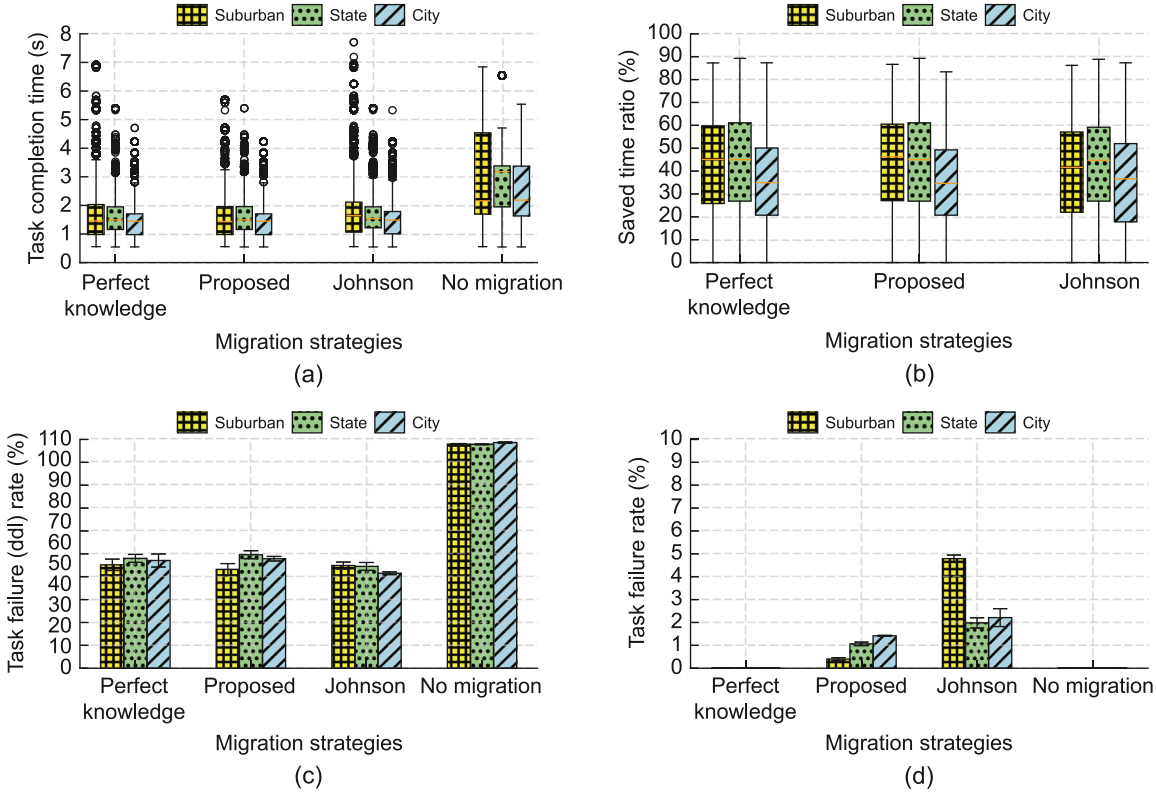


Fig. 6. Migration performance, Nagoya scenario during morning rush hour. (a) Task completion time (b) Saved time ratio (c) Task failure rate due to missed deadline (d) Task failure rate due to mobility prediction.

5.6. Discussion

An effective task migration strategy in dynamic vehicular environments, involving predicted vehicle dwell times, is crucial for reducing task completion time and minimizing failures. Using a shortest-delay-oriented greedy algorithm significantly reduces delays and deadline misses compared to no migration. Overall, our strategy excels by addressing two key challenges: minimizing task delay and reducing the risk of failure due to vehicle mobility.

Accurate dwell time prediction prevents failures due to unpredictable mobility. The proposed random forest model outperforms previous approaches like Johnson's SU distribution, achieving near-optimal dwell time prediction for offloading. Our random forest model enhances task offloading by selecting candidates that minimize completion time and remain in the VMC until task completion. By combining the shortest-delay-oriented algorithm with close-to-accurate dwell time prediction, our strategy optimizes task offloading, ensuring tasks are assigned to vehicles that minimize completion time and remain in the VMC until completion.

Lessons learned from the evaluation are twofold. First, unlike previous work relying on static assumptions, we used realistic urban traffic data for dwell time prediction, significantly reducing failures due to mobility. Second, we evaluated both time efficiency and task failure metrics, resulting in a comprehensive utility function that aligns with real-world efficiency and reliability concerns.

6. Conclusion

In this paper, we presented a task migration mechanism for vehicular micro clouds (VMCs), focusing on minimizing task completion time and eliminating task failures by utilizing a dynamic dwell time prediction model. Our approach estimates the upper-bounded remaining duration for vehicles within VMCs at each time step, letting optimal offloading decisions without results loss. The proposed mechanism is based on two main pillars: (1) selecting the optimal target for task migration based on the shortest estimated task completion time, and (2) ensuring that both the sender and target vehicles remain within the VMC till task completion by leveraging precise dwell time predictions.

We evaluated our approach using a simple intersection scenario as well as two real-world urban mobility scenarios, Luxembourg and Nagoya. The use of diverse traffic data from different urban conditions and various times of the day provided a realistic and comprehensive validation environment for our task migration strategy and dwell time predictions. The results demonstrate significant reductions in task completion times and the elimination of two types of task failures: unpredicted vehicle leaves and deadline misses. Our findings validate both the effectiveness of our task migration mechanism and the accuracy of the dwell time prediction model.

For future work, we plan to explore how task splitting and task dependencies can be integrated to further enhance task migration performance. Also, applicability to highway scenarios with high-speed vehicles needs to be explored.

CRediT authorship contribution statement

Ziqi Zhou: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Agon Memedi:** Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Methodology, Conceptualization. **Chunghan Lee:** Writing – review & editing, Resources, Project administration, Funding acquisition, Conceptualization. **Seyhan Ucar:** Writing – review & editing, Resources, Project administration, Conceptualization. **Onur Altintas:** Writing – review & editing, Supervision, Resources, Project administration, Investigation, Funding acquisition, Conceptualization. **Falko Dressler:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] Christoph Sommer, Falko Dressler, Vehicular networking, Cambridge University Press, ISBN: 978-1-107-04671-9, 2014, http://dx.doi.org/10.1017/CBO9781107110649.

[2] Shaimaa Abdelnabi Abdel Hakeem, Anar Abdel Hady, Hyungwon Kim, 5G-V2X: standardization, architecture, use cases, network-slicing, and edge-computing, ACM/ Springer Wirel. Netw. (ISSN: 1022-0038) (2020) 6015–6041, http://dx.doi.org/10.1007/s11276-020-02419-8.

[3] Rafael Molina-Masegosa, Javier Gozalvez, Miguel Sepulcre, Configuration of the C-V2X mode 4 sidelink PC5 interface for vehicular communication, in: 14th International Conference on Mobile Ad-Hoc and Sensor Networks (MSN 2018), Shenyang, China, 2018, http://dx.doi.org/10.1109/msn.2018.00014.

[4] Pavel Mach, Zdenek Becvar, Mobile edge computing: A survey on architecture and computation offloading, IEEE Commun. Surv. Tutorials (ISSN: 1553-877X) 19 (3) (2017) 1628–1656, http://dx.doi.org/10.1109/COMST.2017.2682318.

[5] Mustafa Emara, Miltiades C. Filippou, Dario Sabella, MEC-assisted end-to-end latency evaluations for C-V2X communications, in: European Conference on Networks and Communications (EuCNC 2018), IEEE, Ljubljana, Slovenia, 2018, http://dx.doi.org/10.1109/eucnc.2018.8442825.

[6] Takamasa Higuchi, Joshua Joy, Falko Dressler, Mario Gerla, Onur Altintas, On the feasibility of vehicular micro clouds, in: 9th IEEE Vehicular Networking Conference (VNC 2017), IEEE, Turin, Italy, (ISSN: 2157-9865) ISBN: 978-1-5386-0986-6, 2017, pp. 179–182, http://dx.doi.org/10.1109/VNC.2017.8275621.

[7] Falko Dressler, Gurjashan Singh Pannu, Florian Hagenauer, Mario Gerla, Takamasa Higuchi, Onur Altintas, Virtual edge computing using vehicular micro clouds, in: IEEE International Conference on Computing, Networking and Communications (ICNC 2019), IEEE, Honolulu, HI, (ISSN: 2325-2626) ISBN: 978-1-5386-9223-3, 2019, http://dx.doi.org/10.1109/ICCNC.2019.8685481.

[8] Falko Dressler, Carla Fabiana Chiasserini, Frank H.P. Fitzek, Holger Karl, Renato Lo Cigno, Antonio Capone, Claudio Ettore Casetti, Francesco Malandrino, Vincenzo Mancuso, Florian Klingler, Gianluca A. Rizzo, V-Edge: Virtual edge computing as an enabler for novel microservices and cooperative computing, IEEE Netw. (ISSN: 1558-156X) 36 (3) (2022) 24–31, http://dx.doi.org/10.1109/MNET.001.2100491.

[9] Ziqi Zhou, Youming Tao, Agon Memedi, Chunghan Lee, Seyhan Ucar, Onur Altintas, Falko Dressler, Optimizing task migration decisions in vehicular edge computing environments, in: 1st IEEE International Conference on Meta Computing (ICMC 2024), IEEE, Qingdao, China, 2024.

[10] Lara Codeca, Raphaël Frank, Thomas Engel, Luxembourg SUMO traffic (LuST) scenario: 24 hours of mobility for vehicular networking research, in: 7th IEEE Vehicular Networking Conference (VNC 2015), IEEE, Kyoto, Japan, (ISSN: 2157-9865) ISBN: 978-1-4673-9411-6, 2015, http://dx.doi.org/10.1109/VNC.2015.7385539.

[11] Takamasa Higuchi, Lei Zhong, Ryokichi Onishi, NUMo: Nagoya urban mobility scenario for city-scale V2X simulations, in: 15th IEEE Vehicular Networking Conference (VNC 2024), IEEE, Kobe, Japan, (ISSN: 2157-9865) 2024, pp. 17–24, http://dx.doi.org/10.1109/VNC61989.2024.10575975.

[12] Tuyen X. Tran, Dario Pompili, Joint task offloading and resource allocation for multi-server mobile-edge computing networks, IEEE Trans. Veh. Technol. (ISSN: 1939-9359) 68 (1) (2019) 856–868, http://dx.doi.org/10.1109/TVT.2018.2881191.

[13] Md Delowar Hossain, Luan N.T. Huynh, Tangina Sultana, Tri D.T. Nguyen, Jae Ho Park, Choong Seon Hong, Eui-Nam Huh, Collaborative task offloading for overloaded mobile edge computing in small-cell networks, in: 34th International Conference on Information Networking (ICOIN 2020), IEEE, Barcelona, Spain, (ISSN: 1976-7684) 2020, pp. 717–722, http://dx.doi.org/10.1109/ICOIN48656.2020.9016452.

[14] Siyao Cheng, Tian Ren, Hao Zhang, Jiayan Huang, Jie Liu, A stackelberg-game-based framework for edge pricing and resource allocation in mobile edge computing, IEEE Internet Things J. (ISSN: 2327-4662) 11 (11) (2024) 20514–20530, http://dx.doi.org/10.1109/JIOT.2024.3372016.

[15] Yuyi Mao, Jun Zhang, Khaled B. Letaief, Dynamic computation offloading for mobile-edge computing with energy harvesting devices, IEEE J. Sel. Areas Commun. (ISSN: 0733-8716) 34 (12) (2016) 3590–3605, http://dx.doi.org/10.1109/JSAC.2016.2611964.

[16] Haitao Zhao, Qixing Zhu, Yue Chen, Yinyang Zhu, A research of task-offloading algorithm for distributed vehicles, in: IEEE International Conference on Communications (ICC 2020), Workshops, Virtual Conference, 2020, pp. 1–5, http://dx.doi.org/10.1109/ICCWorkshops49005.2020.9145331.

[17] Chenhao Wu, Zhongwei Huang, Yuntao Zou, Delay constrained hybrid task offloading of internet of vehicle: A deep reinforcement learning method, IEEE Access (ISSN: 2169-3536) 10 (2022) 102778–102788, http://dx.doi.org/10.1109/ACCESS.2022.3206359.

[18] Hao Qin, Guoping Tan, Siyuan Zhou, Yong Ren, Adaptive learning-based multi-vehicle task offloading, in: IEEE/CIC International Conference on Communications in China (ICCC 2020), IEEE, Chongqing, China, (ISSN: 2377-8644) ISBN: 978-1-7281-7328-3, 2020, pp. 1033–1038, http://dx.doi.org/10.1109/ICCC49849.2020.9238793.

[19] Junhui Zhao, Qiuping Li, Yi Gong, Ke Zhang, Computation offloading and resource allocation for cloud assisted mobile edge computing in vehicular networks, IEEE Trans. Veh. Technol. (ISSN: 1939-9359) 68 (8) (2019) 7944–7956, http://dx.doi.org/10.1109/TVT.2019.2917890.

[20] Narisu Cha, Celimuge Wu, Tsutomu Yoshinaga, Yusheng Ji, Kok-Lim Alvin Yau, Virtual edge: Exploring computation offloading in collaborative vehicular edge computing, IEEE Access (ISSN: 2169-3536) 9 (2021) 37739–37751, http://dx.doi.org/10.1109/access.2021.3063246.

[21] Yufei Zou, Li Lin, Lei Zhang, A task offloading strategy for compute-intensive scenarios in UAV-assisted IoV, in: 5th IEEE International Conference on Electronic Information and Communication Technology (ICEICT 2022), IEEE, Hefei, China, ISBN: 978-1-66547-212-8, 2022, pp. 427–431, http://dx.doi.org/10.1109/ICEICT55736.2022.9909200.

[22] Gurjashan Singh Pannu, Seyhan Ucar, Takamasa Higuchi, Onur Altintas, Falko Dressler, Dwell time estimation at intersections for improved vehicular micro cloud operations, Elsevier Ad Hoc Netw. (ISSN: 1570-8705) 122 (2021) 102606, http://dx.doi.org/10.1016/j.adhoc.2021.102606.

[23] Max Schettler, Gurjashan Singh Pannu, Seyhan Ucar, Takamasa Higuchi, Onur Altintas, Falko Dressler, Learning-based dwell time prediction for vehicular micro clouds, in: 18th IEEE International Conference on Mobility, Sensing and Networking (MSN 2022), IEEE, Guangzhou, China, 2022, pp. 542–549, http://dx.doi.org/10.1109/MSN57253.2022.00091.

[24] Hui Guo, Lan-lan Rui, Zhi-peng Gao, V2V task offloading algorithm with LSTM-based spatiotemporal trajectory prediction model in SVCNs, IEEE Trans. Veh. Technol. (ISSN: 1939-9359) 71 (10) (2022) 11017–11032, http://dx.doi.org/10.1109/TVT.2022.3185085.

[25] Zhiwei Zhang, Zehan Chen, Yulong Shen, Xuewen Dong, Ning Xi, A dynamic task offloading scheme based on location forecasting for mobile intelligent vehicles, IEEE Trans. Veh. Technol. (ISSN: 1939-9359) 73 (6) (2024) 7532–7546, http://dx.doi.org/10.1109/TVT.2024.3351224.

[26] Xiaolong Xu, Chenyi Yang, Muhammad Bilal, Weimin Li, Huihui Wang, Computation offloading for energy and delay trade-offs with traffic flow prediction in edge computing-enabled IoV, IEEE Trans. Intell. Transp. Syst. (ISSN: 1558-0016) 24 (12) (2023) 15613–15623, http://dx.doi.org/10.1109/TITS.2022.3221975.

[27] Baiquan Lv, Chao Yang, Xin Chen, Zhihua Yao, Junjie Yang, Task offloading and serving handover of vehicular edge computing networks based on trajectory prediction, IEEE Access (ISSN: 2169-3536) 9 (2021) 130793–130804, http://dx.doi.org/10.1109/ACCESS.2021.3112077.

[28] Xu Chen, Lei Jiao, Wenzhong Li, Xiaoming Fu, Efficient multi-user computation offloading for mobile-edge cloud computing, IEEE/ACM Trans. Netw. (ISSN: 1063-6692) 24 (5) (2016) 2795–2808, http://dx.doi.org/10.1109/TNET.2015.2487344.

[29] Youngsu Jang, Jinyeop Na, Seongah Jeong, Joonhyuk Kang, Energy-efficient task offloading for vehicular edge computing: Joint optimization of offloading and bit allocation, in: 91st IEEE Vehicular Technology Conference (VTC 2020-Spring), IEEE, Virtual Conference, (ISSN: 2577-2465) ISBN: 978-1-7281-4053-7, 2020, pp. 1–5, http://dx.doi.org/10.1109/VTC2020-Spring48590.2020.9128785.

[30] Yujiong Liu, Shangguang Wang, Qinglin Zhao, Shiyu Du, Ao Zhou, Xiao Ma, Fangchun Yang, Dependency-aware task scheduling in vehicular edge computing, IEEE Internet Things J. (ISSN: 2327-4662) 7 (6) (2020) 4961–4971, http://dx.doi.org/10.1109/JIOT.2020.2972041.

[31] Muhammad Saleh Bute, Pingzhi Fan, Gang Liu, Fakhar Abbas, Zhiguo Ding, A collaborative task offloading scheme in vehicular edge computing, in: 93rd IEEE Vehicular Technology Conference (VTC 2021-Spring), IEEE, Virtual Conference, (ISSN: 2577-2465) 2021, pp. 1–5, http://dx.doi.org/10.1109/VTC2021-Spring51267.2021.9448975.

[32] Jinming Shi, Jun Du, Jingjing Wang, Jian Wang, Jian Yuan, Priority-aware task offloading in vehicular fog computing based on deep reinforcement learning, IEEE Trans. Veh. Technol. (ISSN: 1939-9359) 69 (12) (2020) 16067–16081, http://dx.doi.org/10.1109/TVT.2020.3041929.

[33] Daniel Krajzewicz, Jakob Erdmann, Michael Behrisch, Laura Bieker-Walz, Recent development and applications of SUMO - simulation of urban mobility, Int. J. Adv. Syst. Meas. 5 (3&4) (2012) 128–138.